

Uno stimatore di verosimiglianza per il pile-up

derivazione passo per passo

Note per la tesi – misura m205, rivelatori di luce CUPID

24 settembre 2026

In una frase. Per ogni evento lo stimatore M risponde alla domanda: “è più verosimile che questo evento sia un pile-up di due impulsi — con un ritardo e una ripartizione d’energia qualsiasi, pesati con quanto sono probabili — oppure un impulso singolo?” Si rigettano gli eventi per cui la risposta pende abbastanza verso il pile-up.

Il percorso:

1. si scrive come è fatto un evento (singolo o pile-up) e come è fatto il rumore;
2. si definisce una “distanza” fra un evento e un modello, pesata con il rumore;
3. si impara ad adattare un template con ampiezza e tempo ignoti: è il filtro ottimo;
4. si scrive il template del pile-up e si scopre che basta la correlazione del filtro ottimo;
5. si confronta pile-up e singolo per un ritardo fissato;
6. si passa a tutti i ritardi: il lemma di Neyman–Pearson dice come combinarli per minimizzare il BI.

Nelle sezioni 8 e 10 si vede cosa fa lo stimatore e quanto guadagna; nella sezione 12 si dice da dove viene ogni passaggio.

Alla fine di ogni passaggio c’è la parte di codice che lo realizza (riquadri grigi), copiata dai file del progetto e spiegata riga per riga. Lo stimatore sta tutto nella classe `PileupLikelihood` di `src/pileup_likelihood.py`: il costruttore `__init__` calcola una volta per canale e punto di lavoro tutto ciò che non dipende dall’evento, il metodo `statistic` calcola M per ogni evento. Gli estratti sono in ordine di derivazione, non di file; i commenti del codice originale sono stati accorciati.

1 Il modello dei dati

Si lavora in frequenza. Le quantità:

X_k	trasformata di Fourier dell’evento, alla frequenza angolare ω_k
S_k	trasformata del template (impulso medio, picco = 1) ¹
J_k	densità spettrale del rumore (NPS)
A, t_0	ampiezza totale e tempo di arrivo, <i>ignoti</i>
$\Delta t, r$	ritardo fra i due impulsi e frazione d’energia del secondo

¹Come nel resto del codice, S è la trasformata del template moltiplicato per una finestra di Hann (`compute_H`).

Impulso singolo. Un impulso di ampiezza A che arriva al tempo t_0 è

$$X_k = A S_k e^{-i\omega_k t_0} + N_k. \quad (1)$$

Il fattore $e^{-i\omega t_0}$ è il teorema di traslazione di Fourier: un ritardo nel tempo è una fase in frequenza.

Pile-up. Due impulsi con la *stessa forma*: il primo porta la frazione $1 - r$ dell'energia e arriva a t_0 , il secondo porta r e arriva Δt dopo:

$$X_k = A S_k e^{-i\omega_k t_0} \left[(1 - r) + r e^{-i\omega_k \Delta t} \right] + N_k. \quad (2)$$

Chiamo $\theta = (\Delta t, r)$ i due parametri che distinguono un pile-up dall'altro.

Rumore. Gaussiano e stazionario. La stazionarietà rende indipendenti le diverse frequenze, ciascuna con varianza proporzionale a J_k . Sono esattamente le ipotesi del filtro ottimo: non se ne aggiungono altre.

Nel codice — `src/simulation.py`, `simulate_frequency_pulses`

Il Monte Carlo genera gli eventi esattamente secondo questo modello, direttamente in frequenza:

```
complex_gauss = rng.normal(size=(nsim, M)) + 1j * rng.normal(size=(nsim, M))
noise_freq = complex_gauss * np.sqrt(nps[None, :])           #  $N_k$ 
amplitude = rng.normal(loc=signal_scale, scale=detector_sigma, size=nsim)[: , None]
signal_freq = amplitude * S[None, :]                        #  $A S_k$ 
generated_r = inverse_cdf_ratio2b(rng.random(nsim))         #  $r$ 
generated_dt = rng.uniform(0, dt_max, nsim)                 #  $dt$ 
exp_term = np.exp(-1j * w * generated_dt[: , None])        #  $e^{-i w dt}$ 
phase = (1 - generated_r[: , None]) + generated_r[: , None] * exp_term
fpulses = signal_freq * phase + noise_freq                  # eq. (2)
```

e poi, in `simulate_psd`, si torna nel tempo:

```
pulses = np.fft.ifft(fpulses, axis=1).real.astype(np.float32)
```

- Le prime due righe sono il rumore N_k : un numero complesso gaussiano per frequenza, con varianza proporzionale a J_k (`nps`). Frequenze diverse sono estratte in modo indipendente: è l'ipotesi di rumore stazionario.
- `amplitude` è A ; con `detector_sigma = 0` (come nella campagna) è fissa e uguale all'ampiezza del punto di lavoro.
- `phase` è la parentesi quadra della (2), con r estratto dalla distribuzione dei pile-up $2\nu\beta\beta$ e Δt uniforme fra 0 e `dt_max`. Gli impulsi singoli si ottengono con `dt_max = 0`: allora `phase` vale 1 per ogni r .
- $t_0 = 0$: nel Monte Carlo il primo impulso sta sempre nella posizione del template. Lo stimatore però non lo sa e lo cerca (sezione 3).
- L'ultima riga porta l'evento nel tempo. Lo stimatore riceve `pulses`, come riceverebbe un evento vero, e rifà da sé la FFT. Il `.real` dimezza la varianza del rumore: è la ragione per cui vale la convenzione della (4), verificata sul Monte Carlo.

2 Una distanza pesata con il rumore

L'idea: confrontare l'evento con un modello frequenza per frequenza, ma dando *meno peso alle frequenze più rumorose*, perché lì una differenza significa poco.

Il prodotto scalare e la norma che lo realizzano sono

$$\langle a, b \rangle = \sum_k \frac{\text{Re}(a_k b_k^*)}{J_k}, \quad \|a\|^2 = \langle a, a \rangle, \quad (3)$$

con la somma su tutte le frequenze. È il prodotto scalare ordinario con ogni termine diviso per il rumore di quella frequenza.

Con la normalizzazione della NPS usata nel codice, la proiezione del rumore su una direzione u ha varianza

$$\text{Var} \langle N, u \rangle = \|u\|^2. \quad (4)$$

È la stessa convenzione per cui la risoluzione del filtro ottimo vale $\sigma_{\text{OF}} = 1/\sqrt{\sum_k |S_k|^2/J_k}$; è verificata sul Monte Carlo ($\sigma_{\text{simulata}}/\sigma_{\text{OF}} = 0.99$).

Per un rumore gaussiano con questa proprietà, la probabilità di osservare X se il modello (senza rumore) è m vale

$$\ln p(X | m) = -\frac{1}{2} \|X - m\|^2 + \text{costante}. \quad (5)$$

In parole: *un modello è tanto più verosimile quanto più è vicino all'evento, misurando la distanza in unità di rumore*. Di conseguenza, per due modelli m_1 e m_2 ,

$$\ln \frac{p(X | m_1)}{p(X | m_2)} = \frac{1}{2} \left(\|X - m_2\|^2 - \|X - m_1\|^2 \right) : \quad (6)$$

il logaritmo del rapporto di verosimiglianza è metà del χ^2 che m_1 lascia *in meno* rispetto a m_2 .

Nel codice — `src/pileup_likelihood.py`, `PileupLikelihood.__init__`, passo 1

```
weight = np.abs(S) ** 2 / nps                                # |S|^2 / J: peso di ogni frequenza
pos = np.arange(1, N // 2)                                   # frequenze positive, senza DC e Nyquist
order = pos[np.argsort(weight[pos])[:-1]]                     # dalla piu' pesante alla piu' leggera
n_keep = np.searchsorted(np.cumsum(weight[order]) / weight[pos].sum(), keep) + 1
self.k = np.sort(order[:n_keep])                             # le frequenze che si tengono
```

- La metrica (3) non compare come funzione a sé: compare ogni volta che si divide per `nps`. `weight` è il contributo di ogni frequenza a $\|S\|^2$: dice quanto quella frequenza conta nel distinguere un impulso dal rumore.
- Un segnale reale ha lo spettro simmetrico ($X_{-k} = X_k^*$), quindi la somma su tutte le frequenze della (3) è il termine a frequenza zero più *due volte* la somma sulle frequenze positive. Per questo il codice lavora solo con `pos` e poi moltiplica per 2.
- Le altre righe scartano le frequenze inutili: si ordinano per peso e si tengono le prime fino a raggiungere la frazione `keep` = $1 - 10^{-6}$ del peso totale. Le frequenze scartate hanno template trascurabile rispetto al rumore e non cambiano la distanza; tenerle costerebbe solo tempo di calcolo (sezione 3).

3 Adattare un template con ampiezza e tempo ignoti

Prendiamo un template qualsiasi T , spostato al tempo t e moltiplicato per un'ampiezza $a \geq 0$. Sviluppando il quadrato,

$$\|X - aTe^{-i\omega t}\|^2 = \|X\|^2 - 2a c_T(t) + a^2 \|T\|^2, \quad c_T(t) = \langle X, Te^{-i\omega t} \rangle, \quad (7)$$

dove si è usato $\|Te^{-i\omega t}\| = \|T\|$ (una fase non cambia il modulo).

Per t fissato è una parabola in a . Il minimo sta in $\hat{a} = \max(0, c_T) / \|T\|^2$ (il vincolo $a \geq 0$ vieta ampiezze negative), e lì il χ^2 scende di $\max(0, c_T)^2 / \|T\|^2$. Scegliendo anche il tempo migliore:

$$L_T = \max_t \frac{\max(0, c_T(t))^2}{\|T\|^2} \quad (8)$$

L_T è il χ^2 spiegato dal template: quanta parte dell'evento, in unità di rumore al quadrato, il template riesce a rendere conto. Sostituire le grandezze ignote (a, t) con il loro valore migliore si chiama *profilare* la verosimiglianza.

Il caso dell'impulso singolo è il filtro ottimo. Con $T = S$:

$$y(t) \equiv c_S(t) = \sum_k \frac{X_k S_k^* e^{i\omega_k t}}{J_k}, \quad L_1 = \max_t \frac{y(t)^2}{R(0)}, \quad R(0) = \|S\|^2 = \sum_k \frac{|S_k|^2}{J_k}. \quad (9)$$

$y(t)$ è la *correlazione del filtro ottimo*: si calcola con una sola FFT inversa, ed è la stessa quantità che l'OF massimizza per trovare ampiezza e tempo. Infatti $\hat{A} = y(\hat{t})/R(0)$ è l'ampiezza dell'OF, e

$$L_1 = \left(\frac{\hat{A}}{\sigma_{\text{OF}}} \right)^2 \quad (10)$$

è il quadrato del rapporto segnale/rumore dell'evento visto dal filtro ottimo.

Nel codice — `src/pileup_likelihood.py`, `__init__` passo 2, e il metodo `correlation`

Tutto ciò che nella (9) non dipende dall'evento si prepara una volta sola:

```
self.t = np.round(np.arange(-t_window, t_window + dt_max + step / 2, step), 12)
self.n_t1 = int(round(2 * t_window / step)) + 1 # tempi possibili del PRIMO impulso
wt = np.outer(w[self.k], self.t)
self.cos, self.sin = np.cos(wt), np.sin(wt)
self.filt = (np.conj(S) / nps)[self.k] # S*/J sulle frequenze tenute
self.dc = (np.conj(S[0]) / nps[0]).real
```

e per ogni evento resta:

```
def correlation(self, pulses):
    X = np.fft.fft(np.asarray(pulses, dtype=np.float64), axis=1)
    Z = X[:, self.k] * self.filt[None, :]
    return X[:, [0]].real * self.dc + 2 * (Z.real @ self.cos - Z.imag @ self.sin)
```

- `self.t` è la griglia dei tempi t su cui si legge $y(t)$: da -0.5 a $+1.3$ ms a passi di `step` = 0.01 ms. Il campionamento (0.1 ms) è troppo grosso per ritardi di qualche decimo di ms, quindi y si legge anche fra un campione e l'altro. Per lo stesso motivo non si usa la FFT inversa, che darebbe y solo sui campioni.
- I primi `n_t1` tempi (da -0.5 a $+0.5$ ms) sono quelli in cui si cerca il primo impulso; la griglia prosegue per altri `dt_max` = 0.8 ms perché serviranno i valori di y al tempo del secondo impulso (sezione successiva).

- Z è $X_k S_k^* / J_k$ sulle frequenze tenute. Il fattore $e^{i\omega_k t}$ della (9) si scrive con coseno e seno: $\text{Re}(Z e^{i\omega t}) = \text{Re}Z \cos \omega t - \text{Im}Z \sin \omega t$. Le due moltiplicazioni matriciali (@) calcolano questa somma per tutti gli eventi e tutti i tempi in un colpo solo.
- Il fattore 2 e il termine a frequenza zero (dc) sono la simmetria dello spettro del passo precedente.

Il χ^2 spiegato da un impulso, L_1 , è nel metodo `statistic`:

```
y = self.correlation(pulses)
y1 = y[:, :self.n_t1] # y al tempo del primo impulso
L1 = np.clip(y1.max(axis=1), 0, None) ** 2 / self.R0
```

Il massimo lungo `axis=1` è il $\max_t$; `np.clip(..., 0, None)` è il $\max(0, c)$ della (8), cioè il vincolo $a \geq 0$; `self.R0` è $R(0) = \|S\|^2$, calcolato nel passo successivo. Il massimo si prende *prima* di clip e quadrato: sono operazioni monotone, quindi il valore scelto è lo stesso, ma si scorre l'array una volta sola invece di tre.

4 Il template del pile-up

Per un pile-up con $\theta = (\Delta t, r)$ fissato, il template è dato dalla (2):

$$P_\theta = S \left[(1-r) + r e^{-i\omega \Delta t} \right]. \quad (11)$$

Servono le due quantità della (8). La prima si ottiene dalla linearità del prodotto scalare e dal teorema di traslazione:

$$\begin{aligned} c_{P_\theta}(t) &= (1-r) \langle X, S e^{-i\omega t} \rangle + r \langle X, S e^{-i\omega(t+\Delta t)} \rangle \\ &= (1-r) y(t) + r y(t + \Delta t). \end{aligned} \quad (12)$$

La seconda, usando $|(1-r) + r e^{-i\omega \Delta t}|^2 = (1-r)^2 + r^2 + 2r(1-r) \cos(\omega \Delta t)$:

$$\|P_\theta\|^2 = [(1-r)^2 + r^2] R(0) + 2r(1-r) R(\Delta t), \quad R(\tau) = \sum_k \frac{|S_k|^2 \cos(\omega_k \tau)}{J_k}. \quad (13)$$

$R(\tau)$ è l'autocorrelazione del template nella metrica del rumore: quanto il template somiglia a se stesso spostato di τ . Mettendo insieme:

$$L(\theta) = \max_t \frac{\max(0, (1-r) y(t) + r y(t + \Delta t))^2}{[(1-r)^2 + r^2] R(0) + 2r(1-r) R(\Delta t)} \quad (14)$$

Perché il metodo è praticabile. Tutto dipende dall'evento solo attraverso $y(t)$, la correlazione che l'OF calcola già. $R(\tau)$ dipende solo dal template e dalla NPS: si calcola una volta per canale e punto di lavoro. Provare centinaia di pile-up diversi non costa nessuna FFT in più: servono solo combinazioni di y letta a due tempi.

Nel codice — `src/pileup_likelihood.py`, `__init__` passi 3 e 4

La parte che dipende solo dal template, cioè la (13):

```
wk, pk = w[self.k], weight[self.k]
R = lambda tau: weight[0] + 2 * np.sum(pk * np.cos(wk * tau))
self.R0 = R(0.0)

self.d_steps = np.arange(0, int(round(dt_max / step)) + 1)
self.r = np.asarray(r_values, dtype=float)
self.normP = np.array([(1 - self.r) ** 2 + self.r ** 2] * self.R0
    + 2 * self.r * (1 - self.r) * R(d * step) for d in self.d_steps])
```

- R è l'autocorrelazione $R(\tau)$ della (13), sulle stesse frequenze usate per y e con lo stesso “DC + due volte le positive”.
- I ritardi non si esprimono in secondi ma in *passi della griglia*: `d_steps` va da 0 a 80, cioè Δt da 0 a 0.8 ms a passi di 0.01 ms (81 valori). Così $y(t + \Delta t)$ è semplicemente y letta d posizioni più avanti nell'array.
- `self.r` sono i 19 valori di r da 0.05 a 0.95.
- `self.normP` è la tabella 81×19 di $\|P_\theta\|^2$: una riga per ritardo, una colonna per r .

Nel codice — `src/pileup_likelihood.py`, `statistic`, il ciclo sui ritardi
Per ogni evento, la (12) e la (14):

```
a = (1 - self.r)[None, :, None] * y1[:, None, :]          # (1-r) y(t): non dipende da dt
r3 = self.r[None, :, None]
c = np.empty_like(a)                                       # riusato a ogni dt
for j, d in enumerate(self.d_steps):
    np.multiply(r3, y[:, None, d:d + self.n_t1], out=c)   # r y(t+dt)
    c += a
    L = np.clip(c.max(axis=2), 0, None) ** 2 / self.normP[j][None, :]
```

- a è $(1 - r)y(t)$ per tutti gli eventi, tutti gli r e tutti i tempi t del primo impulso: ha forma (eventi, r , t). Non dipende dal ritardo, quindi si calcola una volta sola, fuori dal ciclo. Gli indici `None` servono solo ad allineare le dimensioni.
- `y[:, None, d:d + self.n_t1]` è $y(t + \Delta t)$: lo stesso array y , letto d posizioni più avanti. Qui si vede perché la griglia dei tempi arriva fino a +1.3 ms.
- Le due righe con `c` scrivono in `c` prima $ry(t + \Delta t)$ e poi ci aggiungono a : il risultato è $cP_\theta(t) = (1 - r)y(t) + ry(t + \Delta t)$, la (12). L'array `c` è sempre lo stesso: non se ne crea uno nuovo per ogni ritardo.
- L'ultima riga è la (14): massimo sul tempo, vincolo di ampiezza positiva, divisione per la norma del template di pile-up (anche qui il massimo prima del quadrato). L ha forma (eventi, r): il χ^2 spiegato da ogni pile-up con questo ritardo.
- Nessuna FFT dentro il ciclo: solo somme e prodotti su y , come promesso dal riquadro sopra.

5 Pile-up contro singolo, per un pile-up fissato

Dalla (6), usando per ciascuna ipotesi il suo adattamento migliore:

$$\ln \frac{\max_{A,t_0} p(X \mid \text{pile-up } \theta)}{\max_{A,t_0} p(X \mid \text{singolo})} = \frac{1}{2} [L(\theta) - L_1]. \quad (15)$$

In parole: *di quanto il miglior pile-up con questi Δt e r spiega l'evento meglio del miglior impulso singolo*. Per $\Delta t = 0$ il template del pile-up coincide con quello singolo e la differenza è zero.

Se si conoscesse in anticipo θ , questa sarebbe la statistica da usare. Ma θ cambia da un pile-up all'altro: serve un modo di combinare tutti i θ .

Nel codice — `src/pileup_likelihood.py`, `statistic`, ultima riga del ciclo

```
terms.append(self.log_prior[None, :] + 0.5 * (L - L1[:, None]))
```

$0.5 * (L - L1[:, \text{None}])$ è esattamente la (15): il logaritmo del rapporto di verosimiglianza fra il miglior pile-up con questo $(\Delta t, r)$ e il miglior impulso singolo, per ogni evento e ogni r . Per $d = 0$ vale zero per ogni r : $y_2 = y_1$, quindi $c = y(t)$, e $\|P_\theta\|^2 = R(0)$. Il logaritmo del peso $\log_prior$ si aggiunge qui, prima della somma della sezione successiva (in forma logaritmica un prodotto diventa una somma).

6 Da un pile-up fissato a tutti: perché la media

6.1 Il lemma di Neyman–Pearson

Lemma di Neyman–Pearson. *Per decidere fra due ipotesi semplici (completamente specificate) H_0 e H_1 , fra tutti i test che rigettano H_0 con la stessa probabilità α quando H_0 è vera, il più potente — quello che, quando è vera H_1 , rigetta H_0 il più spesso possibile — è: rigettare H_0 quando*

$$\frac{p(X | H_1)}{p(X | H_0)} > k, \quad (16)$$

con la soglia k scelta in modo da ottenere proprio la probabilità α [1].

Nel nostro caso H_0 = “impulso singolo”, $\alpha = 0.1$ (si accetta il 90% dei singoli), e “rigettare H_0 ” significa riconoscere l’evento come pile-up. Una dimostrazione breve è in appendice A.

6.2 Il BI è una media sui pile-up

Il BI conta i pile-up che sopravvivono al taglio, *mediati* su come si presentano:

$$\text{BI} = K \int \pi(\theta) P(\text{sopravvive} | \theta) d\theta = K \left[1 - \int \pi(\theta) P(\text{rigettato} | \theta) d\theta \right], \quad (17)$$

dove $\pi(\theta)$ è la distribuzione vera dei pile-up (sezione 6.4). Ora il passaggio chiave. Se $\mathcal{R}$ è l’insieme degli eventi rigettati,

$$\int \pi(\theta) P(\text{rigettato} | \theta) d\theta = \int \pi(\theta) \int_{\mathcal{R}} p(X | \theta) dX d\theta = \int_{\mathcal{R}} \underbrace{\int \pi(\theta) p(X | \theta) d\theta}_{\bar{p}_1(X)} dX, \quad (18)$$

dove si sono solo scambiati gli integrali. La potenza *media* contro tutti i pile-up è quindi la potenza contro **un’unica ipotesi semplice**: “ X è stato generato dalla miscela $\bar{p}_1$ ”, cioè da un pile-up con θ estratto da π . Il lemma allora dice che, a accettazione dei singoli fissata, il test che minimizza il BI è

$$\Lambda(X) = \frac{\bar{p}_1(X)}{p_0(X)} = \int \pi(\theta) \frac{p(X | \theta)}{p_0(X)} d\theta. \quad (19)$$

Questo passaggio è esatto: minimizzare il BI è un problema di Neyman–Pearson contro la miscela. È il criterio della *potenza media pesata* della letteratura statistica [6, 7].

6.3 Ampiezza e tempo ignoti: la statistica M

In entrambe le ipotesi ampiezza e tempo sono ignoti, quindi nessuna delle due è davvero semplice. Si fa la scelta standard: si sostituisce ogni verosimiglianza con il suo massimo su A e t_0 (verosimiglianza profilata [8]). Con la (15), e scrivendo l’integrale come somma su una griglia di valori di θ con pesi $\pi(\theta)$ normalizzati,

$$M = \ln \sum_{\theta} \pi(\theta) \exp \left[\frac{1}{2} (L(\theta) - L_1) \right] \quad (20)$$

Profilare invece di integrare su ampiezza e tempo è un’approssimazione: per questo M è *vicino* al test ottimo, non esattamente il test ottimo.

Perché la media e non il massimo. Il test di verosimiglianza più comune (GLRT) prende il *massimo* su θ invece della media. Risponde a un'altra domanda — quanto somiglia l'evento al pile-up che gli somiglia di più — e non minimizza il BI. Misurato: da solo è peggio del rapporto dei massimi del 9% su ch31, mentre M è meglio del 2%.

6.4 La distribuzione dei pile-up

$\pi(\Delta t, r) = \pi(\Delta t) \pi(r)$:

- Δt **uniforme** fra 0 e $\Delta t_{\max} = 0.8$ ms: le coincidenze casuali sono poissoniane, quindi il secondo evento cade con la stessa probabilità in qualsiasi punto della finestra. $\Delta t_{\max}$ è lo stesso usato dal BI analitico e dal Monte Carlo.
- r secondo la distribuzione della ripartizione d'energia fra i due eventi (`pdf_ratio2b`), la stessa del BI analitico e del Monte Carlo [10].

Se π fosse sbagliata il test resterebbe valido (l'accettanza dei singoli si fissa comunque al 90%), ma non sarebbe più ottimo.

6.5 Nel codice: i pesi e la media

Nel codice — `src/pileup_likelihood.py`, `__init__` passo 5

```
pr = pdf_ratio2b(self.r)
self.log_prior = np.log(pr / pr.sum()) - np.log(len(self.d_steps))
```

- `pdf_ratio2b` (`utility/double_beta_spectrum.py`) è la densità di r , la stessa con cui il Monte Carlo estrae i pile-up. Divisa per la sua somma dà pesi che sommano a 1 sui 19 valori di r .
- Il ritardo è uniforme: ognuno degli 81 valori pesa $1/81$, cioè $-\ln 81$ nel logaritmo. Quindi $\sum_{\theta} \pi(\theta) = 1$ esattamente sulla griglia.
- Si tiene il logaritmo del peso perché la somma si fa in forma logaritmica (sotto).

Nel codice — `src/pileup_likelihood.py`, `statistic`, dopo il ciclo

```
return logsumexp(np.concatenate(terms, axis=1), axis=1)
```

- `terms` contiene, per ogni ritardo, i valori $\ln \pi(\theta) + \frac{1}{2}[L(\theta) - L_1]$ di tutti gli r ; `concatenate` li mette in fila, $81 \times 19 = 1539$ per evento.
- `logsumexp` calcola $\ln \sum e^{(\cdot)}$, cioè esattamente la (20): $\ln \sum_{\theta} \pi(\theta) e^{[L(\theta) - L_1]/2}$. Lo fa sottraendo prima il termine più grande, così l'esponenziale non va fuori scala quando $L(\theta) - L_1$ arriva a centinaia.

Una conseguenza utile per leggere i valori: siccome $\sum \pi = 1$ e per $\Delta t = 0$ il termine vale $e^0 = 1$, M è negativo quando l'evento è spiegato meglio da un singolo che dalla media dei pile-up. Su un singolo senza rumore contribuisce solo la riga $\Delta t = 0$ e $M = \ln(1/81)$.

7 Il taglio e il BI

La distribuzione di M sui singoli non ha una forma chiusa. Si procede esattamente come per il rapporto dei massimi: si calcola M su impulsi singoli simulati, si prende la soglia c che ne lascia passare il 90%, si rigettano gli eventi con $M > c$, e

$$\text{BI} = K \times (\text{frazione di pile-up simulati con } M \leq c). \quad (21)$$

Non c'è niente da addestrare: servono solo il template, la NPS e la soglia.

Nel codice — `simulate_BI_error_m205.py`, `make_likelihood`, `run_pair` e `bi_from`

Lo stimatore si costruisce una volta per punto, sul template di training e sulla NPS:

```
S, w, _ = an.compute_H(template_pulse(channel, wp, f["train"], f["sim"]), nps, np.hanning,
                        sampling_rate=SAMPLING_RATE)
return PileupLikelihood(S, nps, w, dt_max=T_MAX)
```

poi gli *stessi* eventi che passano per i filtri di banda passano anche per M :

```
stats = () if likelihood is None else (lambda p: -likelihood.statistic(p),)
singles = simulate_psd(S, nps, w, filters, signal_amp, 0.0, seed, stats)
pileups = simulate_psd(S, nps, w, filters, signal_amp, T_MAX, seed, stats)
out = [bi_from(s, p) for s, p in zip(singles, pileups)]
```

e il BI si calcola come per il rapporto dei massimi:

```
cut = np.percentile(psd_single, 100 - ACCEPTANCE * 100)
rp = float(np.mean(psd_pileup < cut))
... BI_mc=fn.K * (1.0 - rp) ...
```

- `compute_H` dà lo spettro S del template con la finestra di Hann, la stessa convenzione dei filtri. Il template è quello di *training* (l'AP simulato), mentre gli eventi nascono dal fit: M non conosce la forma vera.
- `T_MAX` = 0.8 ms è lo stesso ritardo massimo del Monte Carlo e del BI analitico.
- Singoli (`dt_max` = 0) e pile-up (`dt_max` = `T_MAX`) sono generati con lo stesso seme: gli eventi sono gli stessi per tutti gli algoritmi, e la differenza di BI fra M e Y si misura evento per evento.
- Si passa $-M$: così il pile-up sta nella coda *bassa* come per il rapporto dei massimi, e `bi_from` vale per entrambi. `cut` è il 10° percentile di $-M$ sui singoli, cioè la soglia c che accetta il 90% dei singoli; `rp` è la frazione di pile-up rigettata e $\text{BI} = K(1 - r_p)$.

8 Cosa fa M : i due regimi

8.1 Ritardo piccolo: M misura la larghezza dell'impulso

Si sviluppa la (12) attorno al *baricentro temporale* $u = t + r\Delta t$ dei due impulsi:

$$(1-r)y(u-r\Delta t) + ry(u+(1-r)\Delta t) = y(u) + \underbrace{\left[-(1-r)r + r(1-r) \right] \Delta t}_{=0} y'(u) + \frac{1}{2} \varepsilon y''(u) + O(\Delta t^3), \quad (22)$$

con

$$\varepsilon = r(1-r) \Delta t^2, \quad (23)$$

la varianza della distribuzione dei tempi dei due impulsi. Il termine al prim'ordine si cancella: **un pile-up molto stretto è, al prim'ordine, solo un impulso spostato**, indistinguibile da un singolo arrivato un po' dopo. Il primo effetto osservabile è un *allargamento* simmetrico,

di entità ε . Nella (13), con $R(\Delta t) = R(0) - \frac{1}{2}M_2\Delta t^2 + O(\Delta t^4)$ e $M_n = \sum_k \omega_k^n |S_k|^2 / J_k$ (quindi $M_0 = R(0)$):

$$\|P_\theta\|^2 = M_0 - \varepsilon M_2 + O(\Delta t^4). \quad (24)$$

Derivando $L = c_P^2 / \|P\|^2$ rispetto a ε in $\varepsilon = 0$:

$$L(\theta) - L_1 = \varepsilon \frac{y}{M_0} \left[y'' + \frac{M_2}{M_0} y \right] + O(\varepsilon^2) = \varepsilon \hat{A} \langle X, g e^{-i\omega t} \rangle + O(\varepsilon^2), \quad (25)$$

dove $\hat{A} = y/M_0$ è l'ampiezza dell'OF e

$$g = S \left(\frac{M_2}{M_0} - \omega^2 \right). \quad (26)$$

Si è usato $y''(t) = \langle X, -\omega^2 S e^{-i\omega t} \rangle$. Il fattore $(M_2/M_0 - \omega^2)$ è positivo sotto la frequenza $f_0 = \sqrt{M_2/M_0}/2\pi$ e negativo sopra: g misura *contenuto a bassa frequenza meno contenuto ad alta frequenza*, bilanciato in modo che su un impulso singolo dia esattamente zero ($\langle S, g \rangle = 0$). Un pile-up ha le alte frequenze smorzate e dà un valore positivo. Normalizzato, $T = \langle X, g e^{-i\omega t} \rangle / \|g\|$ è lo **stimatore di larghezza**, con media 0 e varianza 1 sui singoli.

Il termine al prim'ordine in ε è la derivata della log-verosimiglianza rispetto all'allargamento: il test che usa solo questo termine è il *test del punteggio*, il più potente contro allargamenti piccoli (localmente più potente) [9]. Quindi, a piccoli Δt , M si comporta come lo stimatore di larghezza, e questo coincide in pratica con quello che fanno i filtri di banda addestrati (correlazione 0.95–0.99 sui singoli). Tutti e tre li stanno sul limite teorico: è la regione da cui viene il 78% del BI su ch31 (figura 1b), e *nessuno stimatore può fare meglio*.

8.2 Ritardo grande: M cerca un secondo impulso

Quando i due impulsi sono ben separati, $R(\Delta t) \approx 0$ (i due template spostati non si sovrappongono nella metrica del rumore). Senza rumore, $y(t_0) \approx (1-r) A R(0)$ e $y(t_0 + \Delta t) \approx r A R(0)$, quindi

$$\begin{aligned} L(\theta) &\approx A^2 R(0) [(1-r)^2 + r^2], \\ L_1 &\approx A^2 R(0) \max(r, 1-r)^2 \quad (\text{il singolo si adatta all'impulso maggiore}), \\ L(\theta) - L_1 &\approx A^2 R(0) \min(r, 1-r)^2 = \left(\text{SNR} \cdot \min(r, 1-r) \right)^2. \end{aligned} \quad (27)$$

Riconoscere un pile-up ben separato equivale a rivelare l'impulso più piccolo da solo, con tutto il suo rapporto segnale/rumore; e questo migliora al crescere della separazione. Il rapporto dei massimi invece *satura*: quando gli impulsi si separano, il massimo della banda alta si aggancia all'impulso maggiore e il rapporto smette di dipendere da Δt . È esattamente dove M guadagna (figura 1a).

9 Implementazione

- $y(t)$ si calcola su una griglia fine (passo 0.01 ms), perché il campionamento (0.1 ms) è troppo grosso rispetto ai ritardi in gioco: somma diretta sulle frequenze che portano il 99.9999% del peso $|S|^2/J$.
- t entro ± 0.5 ms dalla posizione nominale; Δt da 0 a 0.8 ms (81 valori); r da 0.05 a 0.95 (19 valori).
- La somma della (20) si calcola in forma logaritmica (`logsumexp`): per pile-up evidenti $L(\theta) - L_1$ arriva a centinaia e l'esponenziale andrebbe fuori scala.

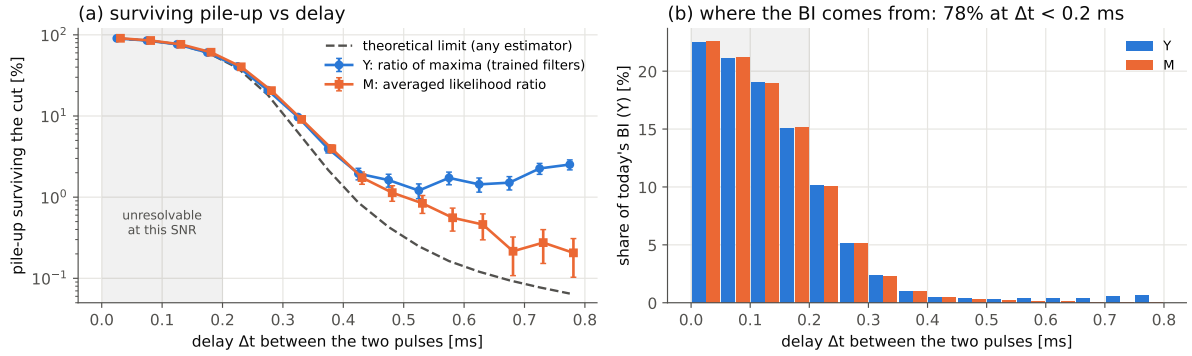


Figura 1: Ch31 WP7, Monte Carlo, 30 000 eventi per popolazione. (a) Frazione di pile-up che sopravvive al taglio in funzione del ritardo: rapporto dei massimi (Y), stimatore M e limite teorico. Oltre ~ 0.45 ms Y smette di migliorare, M continua a scendere. (b) Da dove viene il BI: il 78% da pile-up con $\Delta t < 0.2$ ms, dove Y , M e il limite coincidono.

- Nella verosimiglianza il template è l'AP simulato, lo stesso su cui sono addestrati i filtri di banda; gli eventi del Monte Carlo sono generati dal fit.
- Costo: circa 0.5 ms per evento. Codice: classe `PileupLikelihood` in `src/pileup_likelihood.py`. Nel Monte Carlo della campagna si attiva con `LIKELIHOOD = True` in `simulate_BI_error_m205.py`.
- Controllo: `python src/pileup_likelihood.py` verifica che $y(t)$ coincida con la correlazione del filtro ottimo calcolata con la FFT (entro 10^{-6} senza rumore), che un singolo senza rumore dia $M \leq 0$ e un pile-up senza rumore $M > 0$.
- Un evento analizzato passo per passo, con le grandezze di ogni sezione e la sua posizione nelle distribuzioni di M : `show_likelihood_m205.py`.

10 Risultati

Riferimento: il rapporto dei massimi Y con i filtri Wiener addestrati a 500 passi (campagna `_swna1`). Per ciascuno stimatore il taglio accetta il 90% dei singoli; le differenze sono misurate sugli *stessi eventi*, con errore da bootstrap appaiato.

punto	SNR _{OF}	M vs Y		limite teorico vs Y
		campione indipendente	eventi della campagna	
Ch31 WP7	20	$-2.32 \pm 0.42\%$	$-1.96 \pm 0.32\%$	-5.6%
Ch91 WP15	25	$-1.75 \pm 0.27\%$	$-1.51 \pm 0.23\%$	-5.1%
Ch34 WP15	124	$-0.41 \pm 0.31\%$	$-0.31 \pm 0.23\%$	-2.4%

- **Campione indipendente:** 30 000 eventi per popolazione, semi diversi da quelli della campagna. **Eventi della campagna:** il Monte Carlo ufficiale (seme 1234, 50 000 eventi), rigenerato; Y ne riproduce il BI a tutte le cifre, quindi sono gli stessi eventi.
- Il guadagno è $\sim 2\%$ dove l'SNR è basso e non misurabile dove è alto, dove Y è già a meno del 2.4% dal limite.

- Usando nella verosimiglianza il template *vero* (il fit che genera gli eventi) invece dell'AP simulato, il risultato non cambia (-2.24 , -1.72 , -0.48%): il guadagno non viene dal conoscere la verità.
- **Il limite teorico** è, per ogni θ , la distanza del pile-up dagli impulsi singoli $d(\theta) = \min_{a,t} \|AP_\theta - aSe^{-i\omega t}\|$, con sopravvivenza $\Phi(z_{0.9} - d)$. È calcolato come se si conoscesse θ in anticipo: nessun test singolo lo raggiunge, e il vero ottimo raggiungibile sta fra M e il limite.

11 Ipotesi e limiti

- Rumore gaussiano e stazionario descritto dalla NPS: le stesse ipotesi del filtro ottimo.
- Forma dell'impulso nota e uguale per i due impulsi del pile-up.
- M confronta “un impulso” con “due impulsi a meno di 0.8 ms”: un secondo impulso più lontano non entra in nessuna delle due ipotesi e non viene visto, anche se grande. Misurato (pile-up a Δt fisso, taglio al 90%): su ch31 WP7 ($r = 0.5$) il rigetto scende dal 100% a 1 ms al 35% a 2 ms; su ch34 WP15 regge fino a 5 ms. Lo stesso vale per Y . È coerente con il BI, che conta solo i pile-up entro 0.8 ms (il fattore K), ma presuppone che quelli più separati siano eliminati da altro.
- Ampiezza e tempo sono profilati, non integrati: M è un'approssimazione del test ottimo.
- La soglia si fissa su impulsi singoli simulati; sui dati reali andrebbe calibrata su un campione di singoli, come per Y .
- I risultati sono su tre punti di lavoro: per un numero da tesi va ripetuto su tutti i canali e punti di lavoro con il Monte Carlo a 50 semi.

12 Da dove viene ogni passaggio

passaggio	fonte
Verosimiglianza gaussiana in frequenza, filtro ottimo, σ_{OF}	standard [2, 3, 5]
Adattamento di un template con ampiezza e tempo ignoti (sez. 3)	standard (<i>matched filter</i>) [3, 4]
Lemma di Neyman–Pearson	[1]
Potenza media su $\pi(\theta)$ = potenza contro la miscela; statistica esponenziale-mediata	<i>weighted average power</i> [6, 7]; test di ipotesi composite con parametri casuali [4, 3]
Verosimiglianza profilata	[8]
Test del punteggio / localmente più potente	[9]
Distribuzione della ripartizione d'energia nei pile-up di eventi $2\nu\beta\beta$	[10]
Riduzione del template di pile-up a $y(t)$ e $R(\tau)$ [eq. (12)–(14)]; identificazione del BI con la potenza media; limiti a piccolo e grande Δt applicati al pile-up	derivati in questo lavoro , a partire dai risultati standard sopra

Non è noto a chi scrive un lavoro che applichi esattamente questo stimatore al pile-up nei rivelatori bolometrici, ma non è stato verificato in modo sistematico: prima di presentarlo come nuovo va controllata la letteratura.

A Dimostrazione breve del lemma di Neyman–Pearson

Sia $\phi^*(X) = 1$ se $p_1(X)/p_0(X) > k$ e 0 altrimenti (il test del lemma), e ϕ un qualsiasi altro test con la stessa probabilità di rigettare H_0 quando è vera, $\int \phi p_0 = \int \phi^* p_0 = \alpha$. Per costruzione $(\phi^* - \phi)(p_1 - kp_0) \geq 0$ per ogni X : dove $p_1 > kp_0$ vale $\phi^* = 1 \geq \phi$, altrove $\phi^* = 0 \leq \phi$. Integrando:

$$\int (\phi^* - \phi) p_1 \geq k \int (\phi^* - \phi) p_0 = 0.$$

Quindi $\int \phi^* p_1 \geq \int \phi p_1$: nessun test con lo stesso α rigetta più spesso H_0 quando è vera H_1 .

Riferimenti bibliografici

- [1] J. Neyman, E. S. Pearson, *On the problem of the most efficient tests of statistical hypotheses*, Phil. Trans. R. Soc. A **231** (1933) 289–337.
- [2] E. Gatti, P. F. Manfredi, *Processing the signals from solid-state detectors in elementary-particle physics*, Riv. Nuovo Cimento **9** (1986) 1–146.
- [3] S. M. Kay, *Fundamentals of Statistical Signal Processing, Vol. II: Detection Theory*, Prentice Hall (1998).
- [4] H. L. Van Trees, *Detection, Estimation, and Modulation Theory, Part I*, Wiley (1968).
- [5] C. Alduino *et al.* (CUORE), *Analysis techniques for the evaluation of the neutrinoless double- β decay lifetime in ^{130}Te with the CUORE-0 detector*, Phys. Rev. C **93** (2016) 045503.
- [6] A. Wald, *Tests of statistical hypotheses concerning several parameters when the number of observations is large*, Trans. Amer. Math. Soc. **54** (1943) 426–482.
- [7] D. W. K. Andrews, W. Ploberger, *Optimal tests when a nuisance parameter is present only under the alternative*, Econometrica **62** (1994) 1383–1414.
- [8] G. Cowan, K. Cranmer, E. Gross, O. Vitells, *Asymptotic formulae for likelihood-based tests of new physics*, Eur. Phys. J. C **71** (2011) 1554.
- [9] D. R. Cox, D. V. Hinkley, *Theoretical Statistics*, Chapman and Hall (1974).
- [10] D. M. Chernyak *et al.*, *Random coincidence of $2\nu 2\beta$ decay events as a background source in bolometric $0\nu 2\beta$ decay experiments*, Eur. Phys. J. C **72** (2012) 1989.